Library Management System with Unit Testing

Name: Diksha Chaudhary

Environment Used: .NET CLI with Visual Studio Code on macOS

Course: *(*Advanced C# and Introduction to .Net*)*

Date: *(*22/02/2026*)*

---

1. Objective

The objective of this assignment is to design and implement a simple Library Management System using a class library and to verify its functionality using unit testing. The focus of the assignment is to apply object-oriented programming concepts and to understand how unit testing helps in validating different operations of the system.

---

2. Tools and Technologies Used

- C#
- .NET SDK
- MSTest Framework
- Visual Studio Code (macOS)

---

3. Project Structure

The solution is divided into two separate projects:

- **LibraryManagementSystem**

  This project contains the core classes such as Book, Borrower, and Library where the main functionality is implemented.

- **LibraryManagementSystem.Tests**

  This project contains unit test methods to verify whether each operation in the library system works correctly.

This separation helps in maintaining clean code and makes the application easier to test and manage.

---

4. Implementation Details

The following classes were created in the class library:

Book

This class stores the details of a book such as title, author, and ISBN.It also maintains the availability status of the book.

```
namespace LibraryManagementSystem;

public class Book
{
    public string Title { get; set; }
    public string Author { get; set; }
    public string ISBN { get; set; }
    public bool IsBorrowed { get; private set; }

    public Book(string title, string author, string isbn)
    {
        Title = title;
        Author = author;
        ISBN = isbn;
        IsBorrowed = false;
    }

    public void BorrowBook()
    {
        IsBorrowed = true;
    }

    public void ReturnBook()
    {
        IsBorrowed = false;
    }
}
```

Borrower

This class represents a library user.It stores the borrower's name, library card number, and the list of books borrowed.

```
namespace LibraryManagementSystem;

public class Borrower
{
    public string Name { get; set; }
    public string LibraryCardNumber { get; set; }

    public List<Book> BorrowedBooks { get; } = new();

    public Borrower(string name, string cardNumber)
    {
        Name = name;
        LibraryCardNumber = cardNumber;
    }

    public void BorrowBook(Book book)
    {
        BorrowedBooks.Add(book);
    }

    public void ReturnBook(Book book)
    {
        BorrowedBooks.Remove(book);
    }
}
```

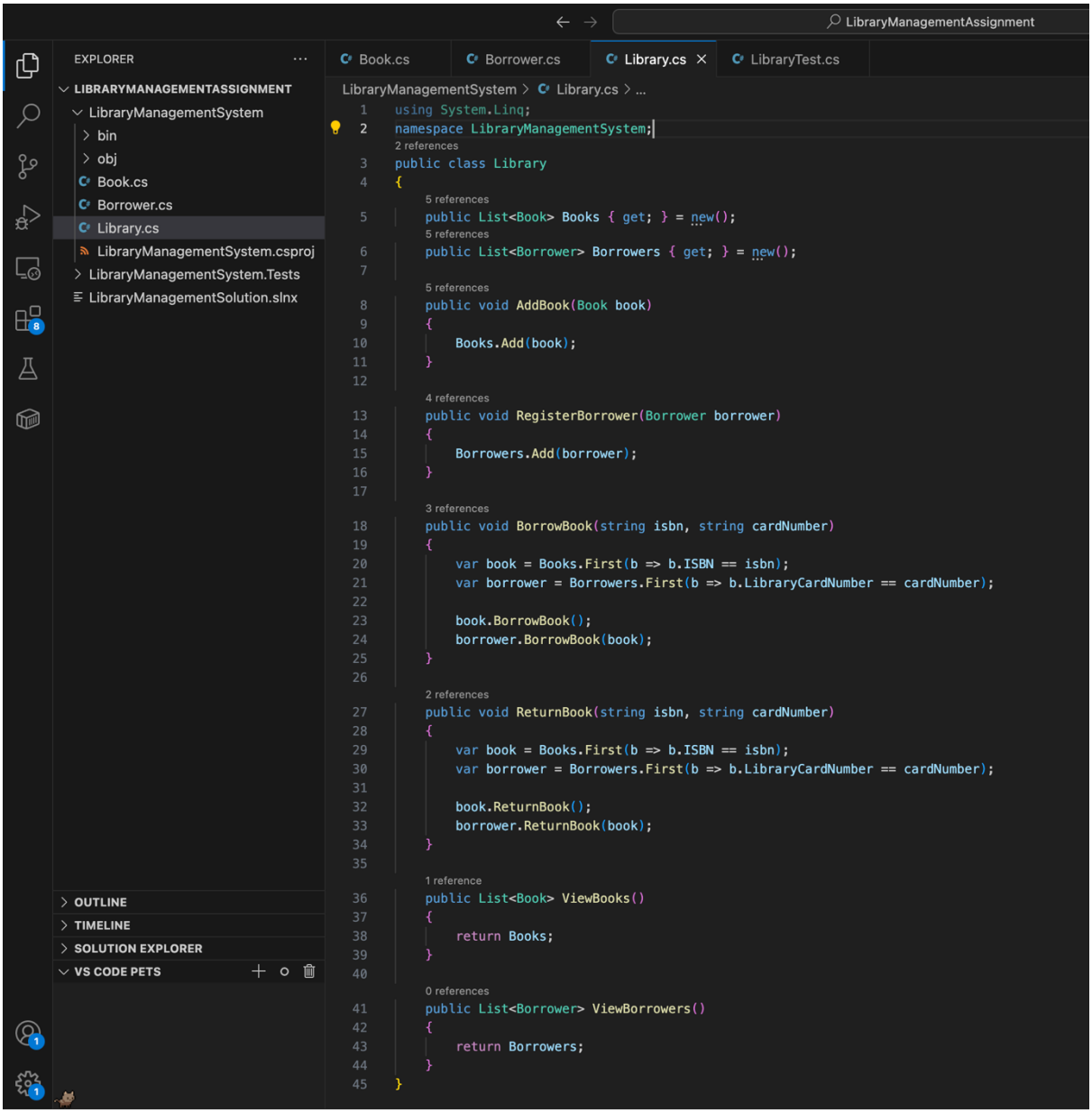
Library

This class manages the complete library operations.

It provides methods to:

- Add a book
- Register a borrower

- Borrow a book
- Return a book
- View all books
- View all borrowers



The borrow and return operations update both the book status and the borrower's borrowed book list.

---

5. Unit Testing

Unit testing is implemented using the MSTest framework to verify all major functionalities of the system.

Test cases were written for:

- Adding a book to the library
- Registering a borrower
- Borrowing a book
- Returning a book
- Viewing the list of books

- Handling invalid operations such as borrowing or returning with incorrect details

Exception handling was also tested to ensure that the system behaves correctly when invalid data is provided.

```
LibraryManagementSystem.Tests > LibraryTest.cs > LibraryTests > ReturnBook_WithInvalidBorrower_ShouldThrowException
using Microsoft.VisualStudio.TestTools.UnitTesting;
using LibraryManagementSystem;
using System;

namespace LibraryManagementSystem.Tests;

[TestClass]
0 references
public class LibraryTests
{
    18 references
    private Library _library = null!;

    [TestInitialize]
    0 references
    public void Setup()
    {
        _library = new Library();
    }

    [TestMethod]
    0 references
    public void AddBook_ShouldAddBookToLibrary()
    {
        var book = new Book("Atomic Habits", "James Clear", "101");

        _library.AddBook(book);

        Assert.HasCount(1, _library.Books, "Book count should be 1 after adding a book.");
    }

    [TestMethod]
    0 references
    public void RegisterBorrower_ShouldAddBorrower()
    {
        var borrower = new Borrower("Diksha", "C1");

        _library.RegisterBorrower(borrower);

        Assert.HasCount(1, _library.Borrowers, "Borrower count should be 1 after registration.");
    }

    [TestMethod]
    0 references
    public void BorrowBook_ShouldMarkBookAsBorrowed()
    {
        var book = new Book("Clean Code", "Robert Martin", "102");
        var borrower = new Borrower("Diksha", "C1");

        _library.AddBook(book);
        _library.RegisterBorrower(borrower);

        _library.BorrowBook("102", "C1");
```

```
LibraryManagementSystem.Tests > LibraryTest.cs > LibraryTests > ReturnBook_ShouldMakeBookAvailable
public class LibraryTests
    public void BorrowBook_ShouldMarkBookAsBorrowed()
        Assert.IsTrue(book.IsBorrowed);
        Assert.HasCount(1, borrower.BorrowedBooks, "Borrowed books count should be 1 after borrowing.");
    }
    [TestMethod]
    0 references
    public void ReturnBook_ShouldMakeBookAvailable()
    {
        var book = new Book("Refactoring", "Martin Fowler", "103");
        var borrower = new Borrower("Diksha", "C1");
        _library.AddBook(book);
        _library.RegisterBorrower(borrower);
        _library.BorrowBook("103", "C1");
        _library.ReturnBook("103", "C1");
        Assert.IsFalse(book.IsBorrowed);
        Assert.IsEmpty(borrower.BorrowedBooks, "Borrowed books count should be 0 after returning.");
    }
    [TestMethod]
    0 references
    public void ViewBooks_ShouldReturnAllBooks()
    {
        _library.AddBook(new Book("Book1", "Author1", "111"));
        var books = _library.ViewBooks();
        Assert.HasCount(1, books, "ViewBooks should return exactly one book.");
    }
    [TestMethod]
    0 references
    public void BorrowBook_WithInvalidISBN_ShouldThrowException()
    {
        var borrower = new Borrower("Diksha", "C1");
        _library.RegisterBorrower(borrower);
        try
        {
            _library.BorrowBook("999", "C1");
            Assert.Fail("Expected InvalidOperationException was not thrown.");
        }
        catch (InvalidOperationException ex)
        {
            Assert.IsInstanceOfType(ex, typeof(InvalidOperationException));
        }
    }
    [TestMethod]
    0 references
    public void ReturnBook_WithInvalidBorrower_ShouldThrowException()
    {
        var book = new Book("Test", "Author", "105");
        _library.AddBook(book);
        try
        {
            _library.ReturnBook("105", "INVALID");
            Assert.Fail("Expected InvalidOperationException was not thrown.");
        }
```

```
LibraryManagementSystem.Tests > LibraryTest.cs > LibraryTests > ReturnBook_ShouldMakeBookAvailable
public class LibraryTests
    public void ReturnBook_WithInvalidBorrower_ShouldThrowException()
        catch (InvalidOperationException ex)
        {
            Assert.IsInstanceOfType(ex, typeof(InvalidOperationException));
        }
    }
}
```

6. Test Execution

All the test cases were executed using the following command: **dotnet test.**

The output shows that all test methods passed successfully.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                                    zsh

(base) dikshachaudhary@Dikshas-MacBook-Air LibraryManagementAssignment % dotnet build
Restore complete (0.8s)
  LibraryManagementSystem net10.0 succeeded (2.4s) → LibraryManagementSystem/bin/Debug/net10.0/LibraryManagementSystem.dll
  LibraryManagementSystem.Tests net10.0 succeeded (0.6s) → LibraryManagementSystem.Tests/bin/Debug/net10.0/LibraryManagementSystem.Tests.dll

Build succeeded in 4.1s
(base) dikshachaudhary@Dikshas-MacBook-Air LibraryManagementAssignment % dotnet test
Restore complete (0.6s)
  LibraryManagementSystem net10.0 succeeded (0.1s) → LibraryManagementSystem/bin/Debug/net10.0/LibraryManagementSystem.dll
  LibraryManagementSystem.Tests net10.0 succeeded (0.2s) → LibraryManagementSystem.Tests/bin/Debug/net10.0/LibraryManagementSystem.Tests.dll
Test Parallelization enabled for /Users/dikshachaudhary/LibraryManagementAssignment/LibraryManagementSystem.Tests/bin/Debug/net10.0/LibraryManagementSystem.Tests.dll (Workers: 8, Scope: MethodLev
el)
  LibraryManagementSystem.Tests test net10.0 succeeded (0.8s)

Test summary: total: 7, failed: 0, succeeded: 7, skipped: 0, duration: 0.8s
Build succeeded in 1.8s
(base) dikshachaudhary@Dikshas-MacBook-Air LibraryManagementAssignment %
```

---

7. Result

All the functionalities of the Library Management System worked as expected.

The unit tests confirmed that both valid and invalid scenarios were handled correctly.

---

8. Conclusion

Through this assignment, I understood how multiple classes can work together to build a real-world application using object-oriented programming. I also learned how unit testing helps in verifying each functionality independently and ensures that the system is reliable and error-free.

This approach improves code quality and makes future modifications easier and safer.